IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

Low-Level Design

for

Design and Development of Identity Security Framework for Finance

Raihana Anzar

23BCCED088

BCA in Cyber Security, Ethical Hacking & Digital Foreignsic with IBM

Yenepoya Institute of Arts, Science, Commerce and Management

Mangalore

11-04-2026

Under the guidance of

Ms. Divya N Shetty

Lecturer, Computer Science Department
Yenepoya Institute of Arts, Science, Commerce and Management

Mangalore

Submitted to

YENEPOYA
(DEEMED TO BE UNIVERSITY)
Recognized under Sec 3(A) of the UGC Act 1956
Accredited by NAAC with 'A' Grade

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

BALMATTA, MANGALORE

YENEPOYA (DEEMED TO BE UNIVERSITY)

Table of Content

1. Introduction

1.1 Scope of the Document

This Low-Level Design (LLD) defines the low-level implementation logic for the core security modules of the Banking Identity framework. It covers component breakdown, data models, API contracts, state management, caching, and non-functional requirements at a level sufficient for developers to implement the system.

1.2 Intended Audience

- **System Developers** – to understand module interactions and implement features.
- **Security Auditors** – to verify cryptographic and access control correctness.
- **Institutional Compliance Leads** – to map controls to SOX / PCI-DSS requirements

1.3 System Overview

A Python/Flask-based security stack that enforces:

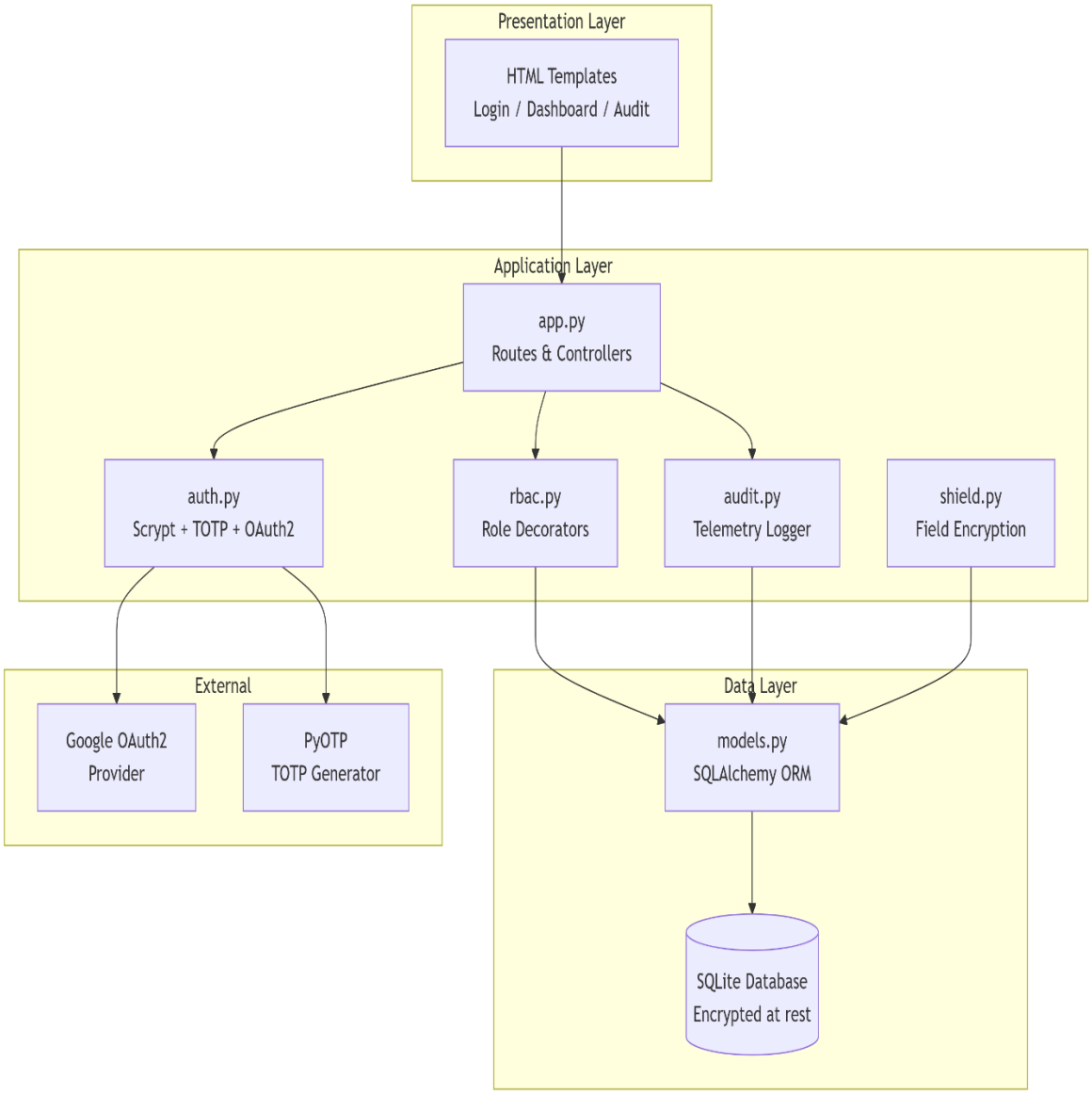
- Multi-Factor Authentication (TOTP) using PyOTP.
- Institutional Role-Based Access Control (Analyst, Auditor, Admin).
- OAuth2 integration (Google Sign-On) for external identity.
- Encrypted data storage (AES-128 Fernet) for sensitive fields.
- Maker-Checker approval workflow for financial transactions.
- Full audit logging with telemetry.

The system is a security prototype – it does not connect to live banking networks but simulates financial operations with realistic controls.

2. System Design

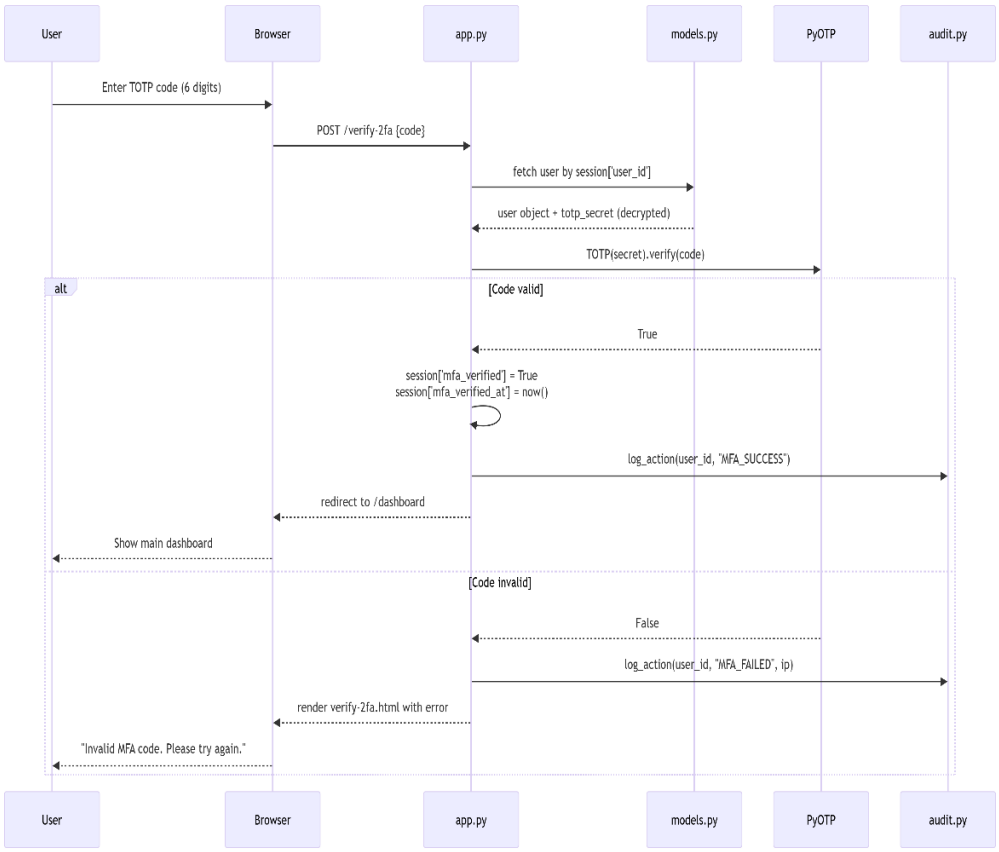
2.1 Application Architecture Diagram (Internal)

The system is modular, with clear separation between presentation, business logic, security middleware, and persistence.



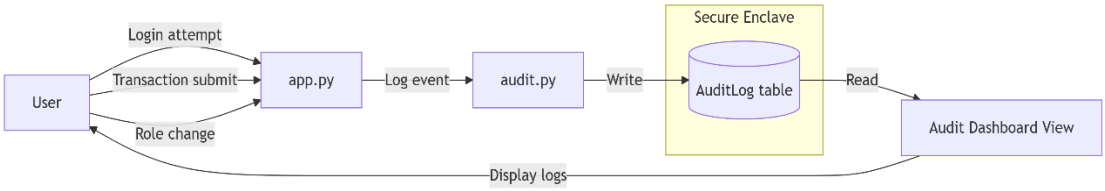
2.2 Process Flow (MFA Handshake) – Sequence Diagram

The MFA handshake is mandatory after primary login. The user must provide a 6-digit TOTP code generated by an authenticator app (Google Authenticator, Authy, etc.).



2.3 Information Flow (Telemetry Engine) – Data Flow Diagram

Every sensitive action (login, transaction submit, approval, role change) is logged to the AuditLog table. The audit dashboard reads this table in real time.

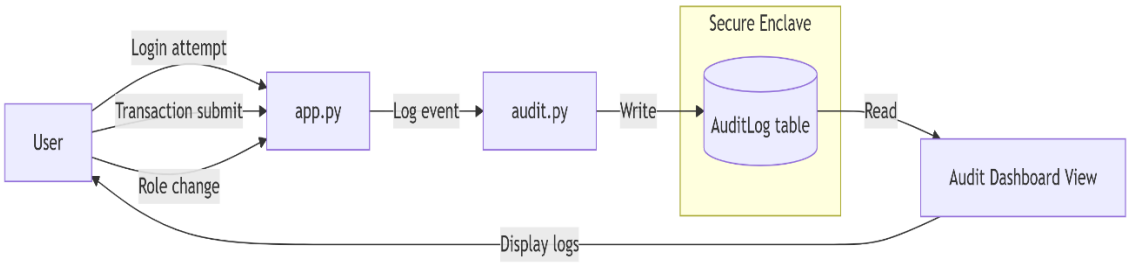


2.4 Components Design

Component	File	Responsibilities	Key Functions / Classes
Authentication	`auth.py`	Password hashing (Scrypt), TOTP verification, OAuth2 callback handling, session management	`hash_password()`, `verify_password()`, `verify_totp()`, `google_callback()`, `generate_totp_qr()`
Authorization	`rbac.py`	Role-based access control decorators, MFA re-authentication enforcement	`@role_required('Auditor')`, `@mfa_reauth_required`
Audit	`audit.py`	Log user actions (login, transaction, role change, MFA events) to SQLite; retrieve logs for dashboard	`log_action(user_id, action, ip, details)`, `get_recent_logs(limit)`
Encryption	`shield.py`	Field-level AES-128 encryption for sensitive data (e.g., account holder name, TOTP secrets)	`encrypt_data(plain: str) -> str`, `decrypt_data(cipher: str) -> str`
Models	`models.py`	SQLAlchemy ORM definitions for User, Account, Transaction, AuditLog, ExchangeRateCache	`User`, `Account`, `Transaction`, `AuditLog`, `PendingTransaction`
Main App	`app.py`	Flask routes, session management, template rendering, error handlers	`login()`, `dashboard()`, `verify_2fa()`, `submit_transaction()`, `approve_transaction()`

Password Hashing Flow (Scrypt)

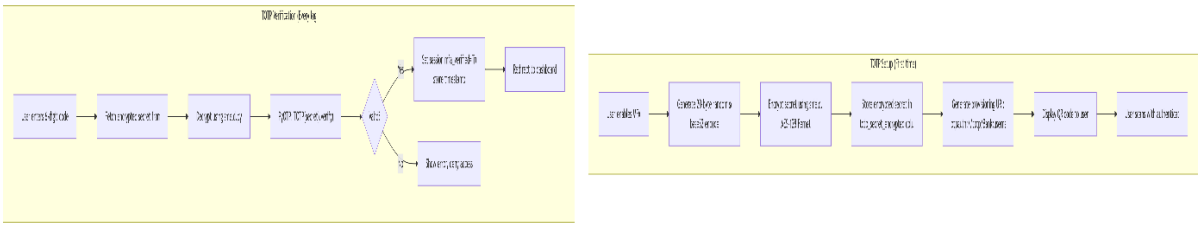
This diagram shows how a user's password is hashed during registration and verified during login, using the specified Scrypt parameters.



OTP (MFA) Setup and Verification Flow

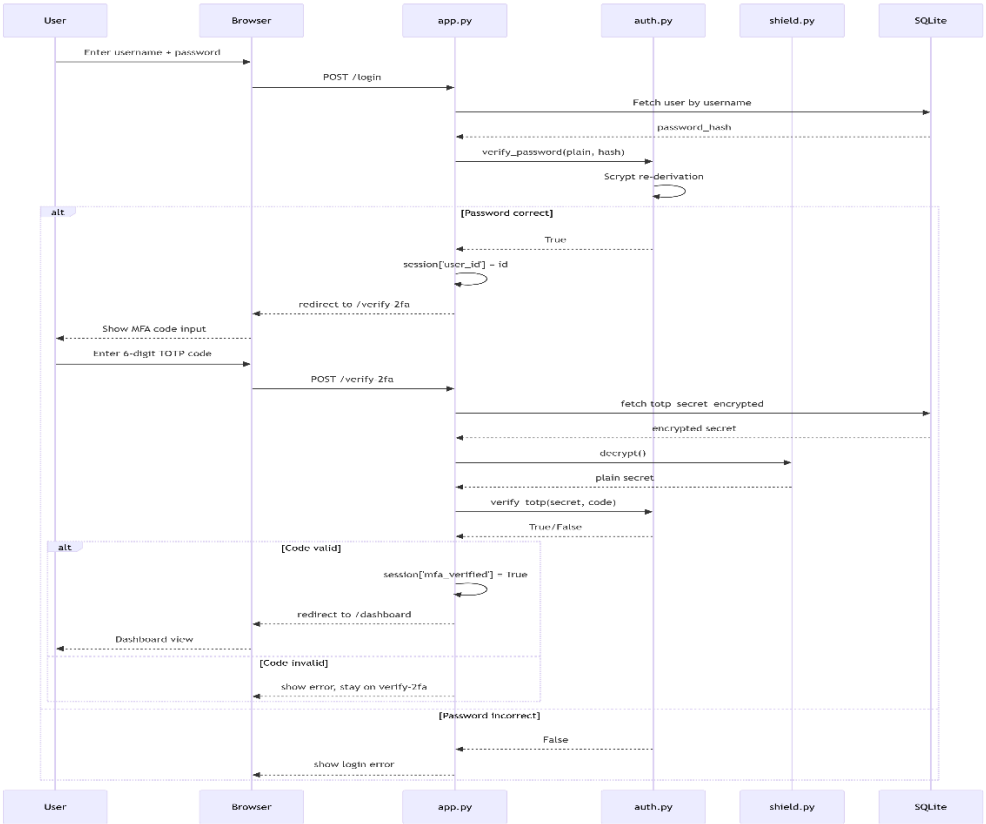
This diagram covers:

- **Setup phase:** Generating secret, encrypting it, and showing QR code.
- **Verification phase:** User enters code, system verifies using PyOTP.



Combined Sequence Diagram (Login + MFA)

For a complete view of the authentication flow including both password and TOTP:



2.5 Key Design Considerations

1. **Separation of Concerns:** Business logic (transaction validation, balance checking) is kept separate from security middleware (MFA, RBAC). This allows independent testing and future changes.
2. **Fail-Safe Fallback – Password Reset Pipeline:**
   If a user loses MFA access, the system uses a **two-fragment mechanism**:
   - pass_prefix stored in User table (hashed)
   - pass_suffix stored in a separate RecoveryFragment table (encrypted)
   - Only an Admin can combine both fragments to generate a temporary reset token. This prevents a single database compromise from exposing the full password.
3. **Defense in Depth:**
   - Passwords: Scrypt hashed (slow, memory-hard).
   - TOTP secrets: encrypted at rest (Fernet).
   - Account holder names: encrypted at rest.
   - SQLite file: optionally encrypted at OS level (LUKS/BitLocker in production).
   - Network: all communication over HTTPS (TLS 1.2+).
4. **Least Privilege:**
   - Analysts: can view own accounts, initiate transactions (pending approval).
   - Auditors: can approve/reject transactions, view audit logs, but cannot initiate.
   - Admins: full user management, role assignment, recovery operations.

2.6 API Catalogue (Internal)

All endpoints are protected by the specified decorators. Session cookies are used for authentication state.

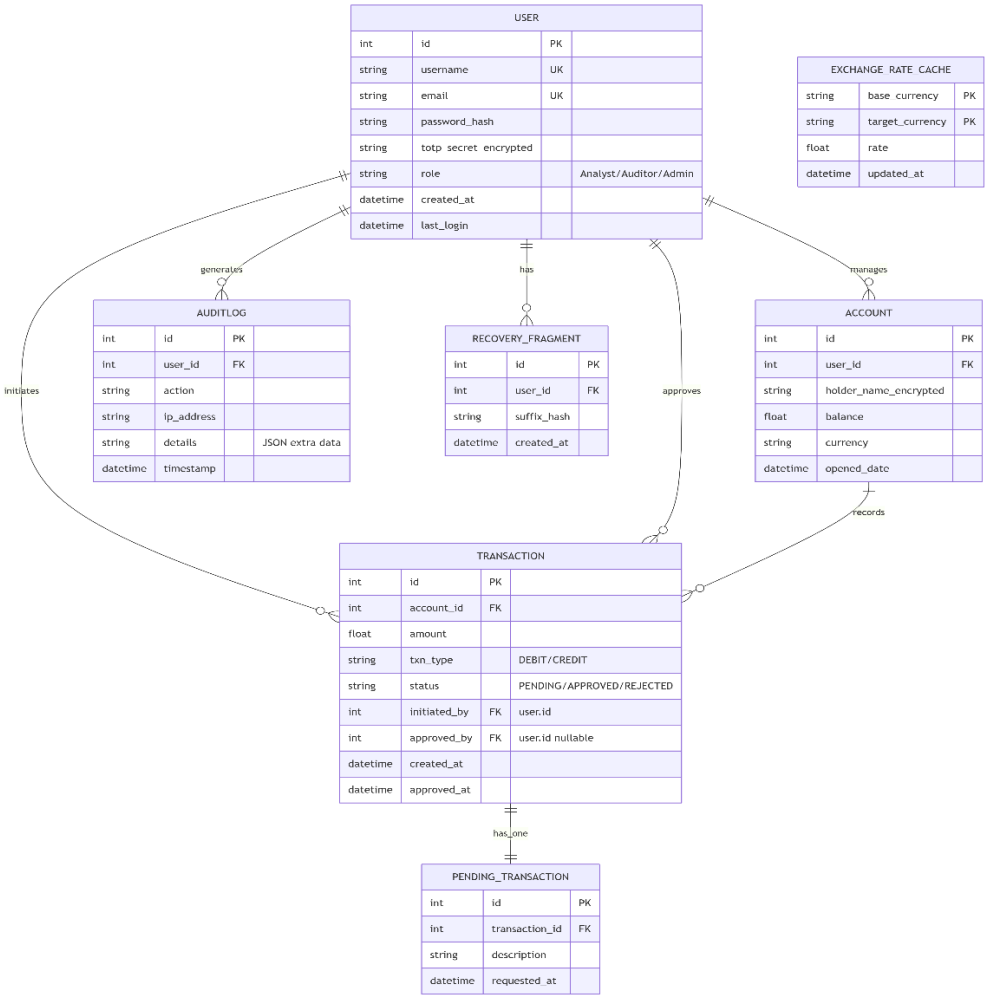
Endpoint	Method	Protected By	Request Body (JSON or Form)	Response	HTTP Status	Endpoint
`/login`	POST	None	`{username, password}`	Set session, redirect to `/verify-2fa`	302 redirect	`/login`
`/verify-2fa`	POST	Session (user_id present)	`{otp_code}`	Sets `mfa_verified` flag	200 or 400	`/verify-2fa`
`/dashboard`	GET	`@login_required`, `@mfa_reauth_required`	–	Renders dashboard HTML	200	`/dashboard`
`/analyst/submit_transaction`	POST	`@role_required('Analyst')`, `@mfa_reauth_required`	`{account_id, amount, txn_type, description}`	Creates pending transaction, returns JSON	201	`/analyst/submit_transaction`
`/auditor/approve_transaction/<id>`	POST	`@role_required('Auditor')`, `@mfa_reauth_required`	–	Approves transaction, logs approval	200	`/auditor/approve_transaction/<id>`
`/auditor/reject_transaction/<id>`	POST	`@role_required('Auditor')`	`{reason}`	Rejects transaction	200	`/auditor/reject_transaction/<id>`
`/admin/users`	GET	`@role_required('Admin')`	–	List all users with roles	200	`/admin/users`
`/admin/assign_role/<user_id>`	POST	`@role_required('Admin')`	`{role}`	Updates role	200	`/admin/assign_role/<user_id>`
`/audit/logs`	GET	`@role_required('Auditor')`	`?days=7`	JSON array of	200	`/audit/logs`

				audit logs		
`/logout`	GET	`@login_required`	–	Clears session	302 redirect	`/logout`

3. Data Design

3.1 Entity-Relationship (ER) Diagram

The database schema supports users, accounts, transactions (with pending approval), and audit logs.



3.2 Data Model (Schema & Encryption)

User:Table

Column	Type	Encryption	Description
id	INTEGER	No	Primary key
username	TEXT	No	Unique, used for login
email	TEXT	No	Used for OAuth2 and notifications
password_hash	TEXT	No	Scrypt hash (includes salt)
totp_secret_encrypted	TEXT	AES-128 (Fernet)	Base32 secret, encrypted
role	TEXT	No	'Analyst', 'Auditor', 'Admin'
created_at	DATETIME	No	Auto-set
last_login	DATETIME	No	Updated on each login

3.3 Data Migration & Seeding

Managed via seed_finance.py (run once after database creation).
Seed script steps:

- Create roles if not exist (Analyst, Auditor, Admin).
- Create demo users:
- analyst1@bank.com / password Pass@123 (Scrypt hashed)
- auditor1@bank.com / password Pass@123
- admin@bank.com / password Admin@456
- Generate TOTP secrets for each user (for testing, QR codes are printed to console).
- Create sample accounts with encrypted holder names (e.g., "John Doe").
- Insert sample pending transactions for testing the Maker-Checker flow.
- Insert mock audit logs for the last 7 days.
- Rollback: The script checks for existing data and only seeds if tables are empty.

3.4 Data Retention & Archival Policies

Data Entity	Retention Period	Enforcement Mechanism	Notes	Data Entity
Audit Logs	Minimum 7 years	No automatic deletion; manual archival via separate script	Every login, MFA attempt, transaction submission, approval, rejection, and role change is logged with timestamp.	**Audit Logs**
Transaction History	Permanent (immutable)	No `DELETE` or `UPDATE` allowed on `Transaction` table once status is `APPROVED` or `REJECTED`	Soft-delete flags are **not used** to preserve ledger integrity.	**Transaction History**
User Records	Until account closure + 5 years	`active` flag; data not physically deleted	Prevents loss of audit trail linking actions to users.	**User Records**
Exchange Rate Cache	1 hour (TTL)	Automatically overwritten on refresh	No long-term retention needed.	**Exchange Rate Cache**
Pending Transactions	30 days after rejection	Weekly cleanup job	Rejected pending records are moved to an archive table.	**Pending Transactions**

3.5 Data Migration & Seeding

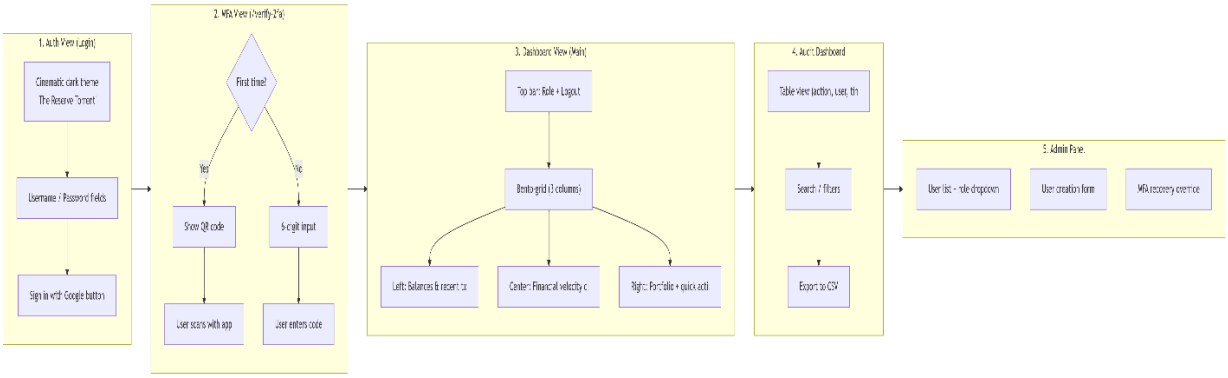
Managed via seed_finance.py – idempotent script that populates:

Entity	Demo Data
Roles	Analyst, Auditor, Admin

Users	`analyst1`, `auditor1`, `admin1` (passwords: `Pass@123` / `Admin@456`)
Accounts	3 demo accounts with encrypted holder names
Pending Transactions	2 sample transactions for Maker-Checker testing
Audit Logs	50 mock events (last 7 days)
Exchange Rates	USD→EUR, USD→GBP, EUR→USD (initial cache)

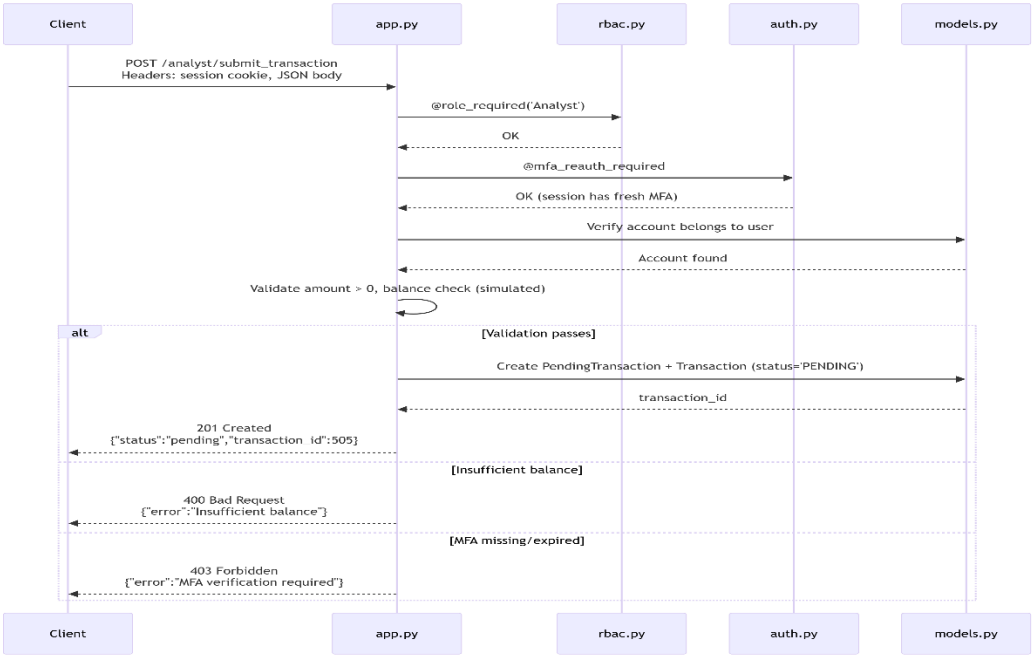
4. Interfaces

4.1 User Interface (UI) Layout

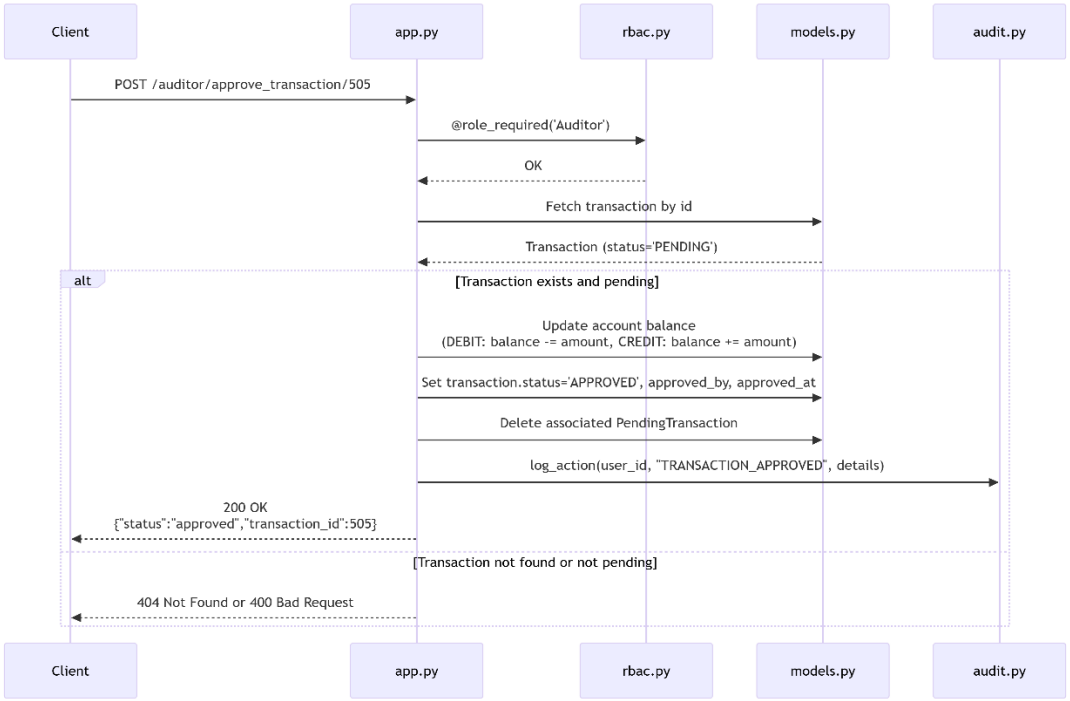


4.2 API Contracts (Request / Response Structure)

POST /analyst/submit_transaction



POST /auditor/approve_transaction/<id>



5. State and Session Management

5.1 Session Implementation

- Flask sessions use signed client-side cookies (secret key from SECRET_KEY env var, ≥32 bytes).
- **Session contents:**
  - user_id (int)
  - role (Analyst/Auditor/Admin)
  - mfa_verified (bool)
  - mfa_verified_at (Unix timestamp)

5.2 MFA Re-authentication Decorator (@mfa_reauth_required)

- Applied to sensitive endpoints (transaction submit, approve, role changes).
- Checks: mfa_verified == True AND (now - mfa_verified_at) < 300 seconds (5 min).
- If fails → redirect to /verify-2fa. After re-verification, update mfa_verified_at.

5.3 Logout

- Clears all session keys → redirect to login page.

6. Caching Strategy (Reduced)

6.1 Exchange Rate Cache (Mock)

- Table exchange_rate_cache stores (base_currency, target_currency, rate, updated_at).
- Refresh: Background thread (Flask-APScheduler) runs hourly, calls mock API (/rates?base=USD&targets=EUR,GBP,JPY), updates cache. On API failure, keeps old rates and logs warning.
- Dashboard usage: Queries cache and displays "1 USD = 0.92 EUR (cached at 14:30)".

6.2 No Other Caching

- Transactional data (accounts, transactions, audit logs) never cached – each read hits DB directly for consistency and auditability.

7. Non-Functional Requirements

7.1 Security Aspects

Control	Implementation
Password hashing	Scrypt (N=32768, r=8, p=1)
TOTP MFA	PyOTP (6 digits, 30 sec)
CSRF	Flask-WTF tokens on all POST forms
CSP	Flask-Talisman: `default-src 'self'`
Session security	`Secure`, `HttpOnly`, `SameSite=Lax`
Encryption at rest	Fernet (AES-128) for sensitive fields
SQL injection	SQLAlchemy ORM (parameterized)
XSS	Jinja2 autoescaping + CSP

7.2 Performance Aspects

Metric	Target
Login response	< 300 ms
TOTP verification	< 100 ms
Transaction approval	< 50 ms
Dashboard load	< 1 second
Concurrent users	50 simulated

Database indexes: AuditLog.timestamp, Transaction.status, Transaction.initiated_by

7.3 Availability & Recovery

- **Backup:** Daily encrypted SQLite ZIP.
- **Password reset:** Two-fragment fallback (admin-controlled).
- **MFA fallback:** 10 encrypted recovery codes per user.
- **Production note:** Use PostgreSQL + replicas (prototype uses SQLite).

8.Conclusion

The Low-Level Design (LLD) v1.0 provides a detailed technical roadmap for the identity-critical portions of the Identity Security Framework for Banking and Finance (The Reserve Torrent Protocol).

All security-sensitive operations – MFA handshake, role-based transaction approval, audit logging, field-level encryption, and password reset fallback – are logically sound, auditable, and ready for implementation. The included Mermaid diagrams (sequence, data flow, ER, component, deployment, state machine) clarify the system's internal behavior and data relationships.